Тема: Використання матричної клавіатури для введення інформації у інтегрованому середовищі PSoC® Creator™

Мета: Ознайомитися з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набути навичок розробки проєкту в інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів та програмної обробки введених даних.

1. Скриншоти проєкту PSoC Creator

1.1. Схема проєкту (TopDesign.cysch)

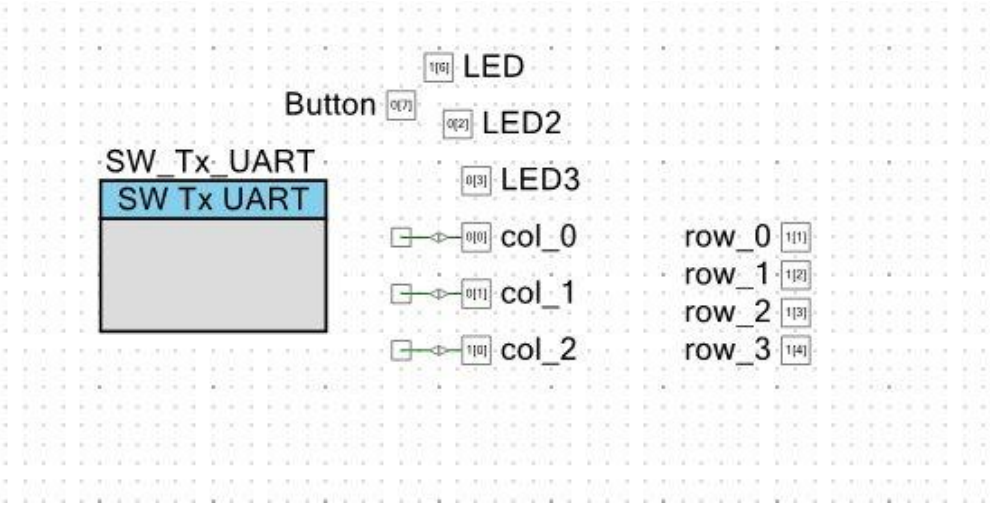

Рисунок 1 – Схема проєкту TopDesign.cysch

1.2. Налаштування ресурсів (Pins .cydwr)

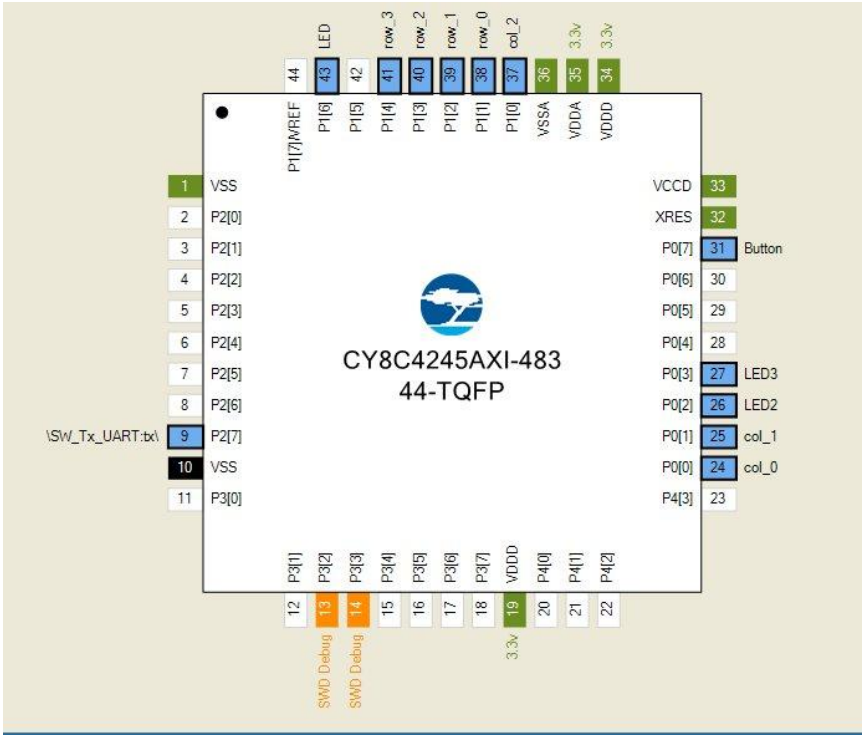

Рисунок 2 – Призначення пінів (CY8C4245AXI-483 44-TQFP)

2. Завдання 1 – Керування LED клавіатурою (if-else)

Реалізовано опитування 3×4 матричної клавіатури. При натисканні кнопки світлодіод змінює колір відповідно до Таблиці 1. При відпусканні кнопки – вмикається білий колір (White).

Колір	Кнопки
Red	1, 7
Green	2, 8
Blue	3, 9
Yellow	4, *
Purple	5, 0
Cyan	6, #

Таблиця 1 – Відповідність кнопок і кольорів

2.1. Код програми main.c (Завдання 1)

```
#include "project.h"

static void (*col_x_SetDriveMode[3])(uint8_t mode) = {
    col_0_SetDriveMode, col_1_SetDriveMode, col_2_SetDriveMode
};
static void (*col_x_Write[3])(uint8_t value) = {
    col_0_Write, col_1_Write, col_2_Write
};
static uint8 (*row_x_Read[4])() = {
    row_0_Read, row_1_Read, row_2_Read, row_3_Read
};

static uint8_t keys[4][3];
static const uint8_t keyMap[4][3] = {
    { 1,  2,  3},
    { 4,  5,  6},
    { 7,  8,  9},
    {10,  0, 11}
};

static void initMatrix() {
    for(int i = 0; i < 3; i++)
        col_x_SetDriveMode[i](col_0_DM_DIG_HIZ);
}

static void readMatrix() {
    for(int col = 0; col < 3; col++) {
        col_x_SetDriveMode[col](col_0_DM_STRONG);
        col_x_Write[col](0);
        for(int row = 0; row < 4; row++)
            keys[row][col] = row_x_Read[row]();
        col_x_SetDriveMode[col](col_0_DM_DIG_HIZ);
    }
```

```
}

static void printMatrix() {
    SW_Tx_UART_PutCRLF();
    for(int row = 0; row < 4; row++) {
        for(int col = 0; col < 3; col++) {
            SW_Tx_UART_PutHexInt(keys[row][col]);
            SW_Tx_UART_PutString(" ");
        }
        SW_Tx_UART_PutCRLF();
    }
    SW_Tx_UART_PutCRLF();
}

static uint8_t getPressedKey() {
    for(int row = 0; row < 4; row++)
        for(int col = 0; col < 3; col++)
            if(keys[row][col] == 0) return keyMap[row][col];
    return 0xFF;
}

static void setLED(uint8_t r, uint8_t g, uint8_t b) {
    LED_Write(r); LED2_Write(g); LED3_Write(b);
}

int main(void) {
    CyGlobalIntEnable;
    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART - Task 1 (if-
else)");
    SW_Tx_UART_PutCRLF();
    initMatrix();
    setLED(0, 0, 0); /* White */

    uint8_t last_key = 0xFF, key_released = 1;

    for(;;) {
        readMatrix();
        uint8_t pressed = getPressedKey();

        if(pressed == 0xFF) {
            if(!key_released) {
                key_released = 1; last_key = 0xFF;
                SW_Tx_UART_PutString("Button released");
                SW_Tx_UART_PutCRLF();
                setLED(1, 1, 1); /* Black */
                setLED(0, 0, 0); /* White */
            }
        } else if(key_released || pressed != last_key) {
            key_released = 0; last_key = pressed;

            if(pressed == 1 || pressed == 7) {
```

```
                SW_Tx_UART_PutString("Button pressed -> Red");
                setLED(0, 1, 1); /* Red: R=ON, G=OFF, B=OFF */
            } else if(pressed == 2 || pressed == 8) {
                SW_Tx_UART_PutString("Button pressed -> Green");
                setLED(1, 0, 1);
            } else if(pressed == 3 || pressed == 9) {
                SW_Tx_UART_PutString("Button pressed -> Blue");
                setLED(1, 1, 0);
            } else if(pressed == 4 || pressed == 10) {
                SW_Tx_UART_PutString("Button pressed ->
Yellow");
                setLED(0, 0, 1);
            } else if(pressed == 5 || pressed == 0) {
                SW_Tx_UART_PutString("Button pressed ->
Purple");
                setLED(0, 1, 0);
            } else if(pressed == 6 || pressed == 11) {
                SW_Tx_UART_PutString("Button pressed -> Cyan");
                setLED(1, 0, 0);
            }
            SW_Tx_UART_PutCRLF();
            printMatrix();
        }
    }
}
```

2.2. Результат виконання в консолі (Завдання 1)

Нижче наведено скриншоти терміналу при натисканні різних кнопок:

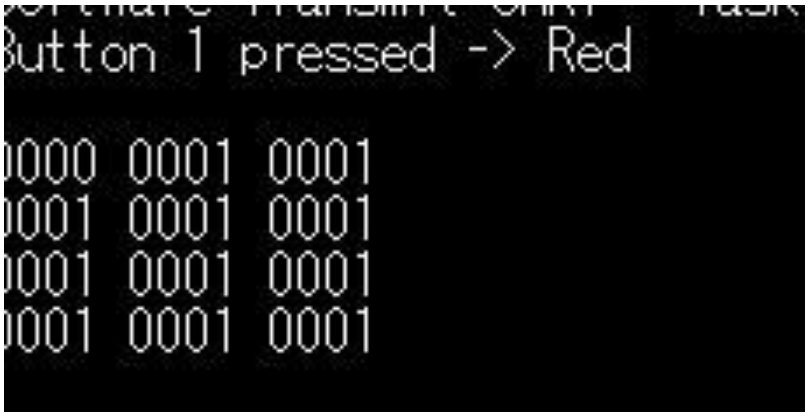

Рисунок 3 – Кнопка 1 → Red

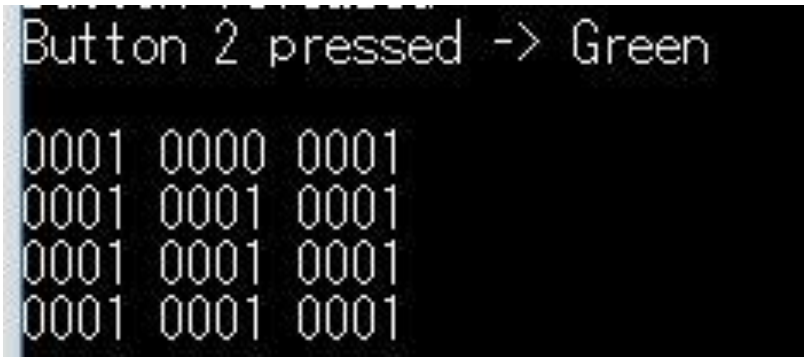

Рисунок 4 – Кнопка 2 → Green

```
Button 3 pressed -> Blue

0001 0001 0000
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button released
```

Рисунок 5 – Кнопка 3 → Blue

```
Button 4 pressed -> Yellow

0001 0001 0001
0000 0001 0001
0001 0001 0001
0001 0001 0001

Button released
```

Рисунок 6 – Кнопка 4 → Yellow

```
Button 5 pressed -> Purple

0001 0001 0001
0001 0000 0001
0001 0001 0001
0001 0001 0001

Button released
```

Рисунок 7 – Кнопка 5 → Purple

```
Button 6 pressed -> Cyan

0001 0001 0001
0001 0001 0000
0001 0001 0001
0001 0001 0001

Button released
```

Рисунок 8 – Кнопка 6 → Cyan

```
Button 7 pressed -> Red

0001 0001 0001
0001 0001 0001
0000 0001 0001
0001 0001 0001

Button released
```

Рисунок 9 – Кнопка 7 → Red

```
Button 8 pressed -> Green

0001 0001 0001
0001 0001 0001
0001 0000 0001
0001 0001 0001

Button released
```

Рисунок 10 – Кнопка 8 → Green

```
Button 9 pressed -> Blue

0001 0001 0001
0001 0001 0001
0001 0001 0000
0001 0001 0001

Button released
```

Рисунок 11 – Кнопка 9 → Blue

```
Button * pressed -> Yellow

0001 0001 0001
0001 0001 0001
0001 0001 0001
0000 0001 0001

Button released
```

*Рисунок 12 – Кнопка * → Yellow*

```
Button 0 pressed -> Purple

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0000 0001

Button released
```

Рисунок 13 – Кнопка 0 → Purple

```
Button # pressed -> Cyan

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0000

Button released
```

Рисунок 14 – Кнопка # → Cyan

3. Завдання 2 – Керування LED (switch-case)

Реалізовано той самий алгоритм керування кольором LED, але з використанням конструкції switch(case) замість if-else. Логіка роботи та результати виконання ідентичні завданню 1.

3.1. Код програми main.c (Завдання 2)

```
#include "project.h"

static void (*col_x_SetDriveMode[3])(uint8_t mode) = {
    col_0_SetDriveMode, col_1_SetDriveMode, col_2_SetDriveMode
};
static void (*col_x_Write[3])(uint8_t value) = {
    col_0_Write, col_1_Write, col_2_Write
};
static uint8 (*row_x_Read[4])() = {
    row_0_Read, row_1_Read, row_2_Read, row_3_Read
};

static uint8_t keys[4][3];
static const uint8_t keyMap[4][3] = {
    { 1,  2,  3},
    { 4,  5,  6},
    { 7,  8,  9},
    {10,  0, 11}
};

static void initMatrix() {
    for(int i = 0; i < 3; i++)
        col_x_SetDriveMode[i](col_0_DM_DIG_HIZ);
}

static void readMatrix() {
    for(int col = 0; col < 3; col++) {
        col_x_SetDriveMode[col](col_0_DM_STRONG);
        col_x_Write[col](0);
        for(int row = 0; row < 4; row++)
            keys[row][col] = row_x_Read[row]();
        col_x_SetDriveMode[col](col_0_DM_DIG_HIZ);
    }
}

static void printMatrix() {
    SW_Tx_UART_PutCRLF();
    for(int row = 0; row < 4; row++) {
        for(int col = 0; col < 3; col++) {
            SW_Tx_UART_PutHexInt(keys[row][col]);
            SW_Tx_UART_PutString(" ");
        }
        SW_Tx_UART_PutCRLF();
    }
    SW_Tx_UART_PutCRLF();
}
```

```
static uint8_t getPressedKey() {
    for(int row = 0; row < 4; row++)
        for(int col = 0; col < 3; col++)
            if(keys[row][col] == 0) return keyMap[row][col];
    return 0xFF;
}

static void setLED(uint8_t r, uint8_t g, uint8_t b) {
    LED_Write(r); LED2_Write(g); LED3_Write(b);
}

int main(void) {
    CyGlobalIntEnable;
    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART - Task 2
(switch-case)");
    SW_Tx_UART_PutCRLF();
    initMatrix();
    setLED(0, 0, 0); /* White */

    uint8_t last_key = 0xFF, key_released = 1;

    for(;;) {
        readMatrix();
        uint8_t pressed = getPressedKey();

        if(pressed == 0xFF) {
            if(!key_released) {
                key_released = 1; last_key = 0xFF;
                SW_Tx_UART_PutString("Button released");
                SW_Tx_UART_PutCRLF();
                setLED(1, 1, 1); setLED(0, 0, 0);
            }
        } else if(key_released || pressed != last_key) {
            key_released = 0; last_key = pressed;

            SW_Tx_UART_PutString("Button ");
            switch(pressed) {
                case 10: SW_Tx_UART_PutString("*"); break;
                case 11: SW_Tx_UART_PutString("#"); break;
                default: { char buf[2]={'0'+pressed,'\0'};
SW_Tx_UART_PutString(buf); break; }
            }
            SW_Tx_UART_PutString(" pressed -> ");

            switch(pressed) {
                case 1: case 7:
                    SW_Tx_UART_PutString("Red");
setLED(0,1,1); break;
                case 2: case 8:
```

```
                    SW_Tx_UART_PutString("Green");
setLED(1,0,1); break;
                case 3: case 9:
                    SW_Tx_UART_PutString("Blue");
setLED(1,1,0); break;
                case 4: case 10:

SW_Tx_UART_PutString("Yellow");setLED(0,0,1); break;
                case 5: case 0:

SW_Tx_UART_PutString("Purple");setLED(0,1,0); break;
                case 6: case 11:
                    SW_Tx_UART_PutString("Cyan");
setLED(1,0,0); break;
                default:
                    SW_Tx_UART_PutString("Unknown"); break;
            }
            SW_Tx_UART_PutCRLF();
            printMatrix();
        }
    }
}
```

3.2. Результат виконання в консолі (Завдання 2)

Результати виконання програми аналогічні завданню 1 (скриншоти наведено вище, рис. 3–14). Програма коректно відображає назву кольору та матрицю стану після кожного натиснення.

4. Завдання 3 – Введення паролю

Реалізовано систему введення паролю за допомогою матричної клавіатури. Пароль складається з 4 символів (кнопки 1-2-3-4). Клавіша * виконує функцію Backspace, клавіша # підтверджує введення. При правильному паролі виводиться "Access allowed", при неправильному – "Access denied".

4.1. Код програми main.c (Завдання 3)

```
#include "project.h"
#include <string.h>

static void (*col_x_SetDriveMode[3])(uint8_t mode) = {
    col_0_SetDriveMode, col_1_SetDriveMode, col_2_SetDriveMode
};
static void (*col_x_Write[3])(uint8_t value) = {
    col_0_Write, col_1_Write, col_2_Write
};
static uint8 (*row_x_Read[4])() = {
    row_0_Read, row_1_Read, row_2_Read, row_3_Read
};

static uint8_t keys[4][3];
static const uint8_t keyMap[4][3] = {
    { 1,  2,  3},
    { 4,  5,  6},
    { 7,  8,  9},
    {10,  0, 11}
};

#define MAX_INPUT_LEN  16
static const uint8_t PASSWORD[] = {1, 2, 3, 4}; /* пароль: 1-2-
3-4 */
static const uint8_t PASS_LEN   = 4u;

static uint8_t inputBuf[MAX_INPUT_LEN];
static uint8_t inputLen = 0;

static void initMatrix() {
    for(int i = 0; i < 3; i++)
        col_x_SetDriveMode[i](col_0_DM_DIG_HIZ);
}

static void readMatrix() {
    for(int col = 0; col < 3; col++) {
        col_x_SetDriveMode[col](col_0_DM_STRONG);
        col_x_Write[col](0);
        for(int row = 0; row < 4; row++)
            keys[row][col] = row_x_Read[row]();
        col_x_SetDriveMode[col](col_0_DM_DIG_HIZ);
    }
}
```

```
static uint8_t getPressedKey() {
    for(int row = 0; row < 4; row++)
        for(int col = 0; col < 3; col++)
            if(keys[row][col] == 0) return keyMap[row][col];
    return 0xFF;
}

static void setLED(uint8_t r, uint8_t g, uint8_t b) {
    LED_Write(r); LED2_Write(g); LED3_Write(b);
}

static void printInputLine() {
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Input: ");
    for(uint8_t i = 0; i < inputLen; i++)
        SW_Tx_UART_PutString("*");
}

static void checkPassword() {
    uint8_t ok = (inputLen == PASS_LEN);
    if(ok)
        for(uint8_t i = 0; i < PASS_LEN; i++)
            if(inputBuf[i] != PASSWORD[i]) { ok = 0; break; }

    if(ok) {
        SW_Tx_UART_PutString("Access allowed");
        SW_Tx_UART_PutCRLF();
        setLED(1, 0, 1); /* Green */
        CyDelay(1500);
    } else {
        SW_Tx_UART_PutString("Access denied");
        SW_Tx_UART_PutCRLF();
        setLED(0, 1, 1); /* Red */
        CyDelay(1500);
    }
    inputLen = 0;
    setLED(0, 0, 0);
    printInputLine();
}

int main(void) {
    CyGlobalIntEnable;
    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART - Task 3
(Password)");
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("* = Backspace  |  # = Confirm");
    SW_Tx_UART_PutCRLF();
    initMatrix();
    setLED(0, 0, 0);
    printInputLine();
```

```
    uint8_t last_key = 0xFF, key_released = 1;

    for(;;) {
        readMatrix();
        uint8_t pressed = getPressedKey();

        if(pressed == 0xFF) {
            if(!key_released) { key_released = 1; last_key =
0xFF; }
        } else if(key_released && pressed != last_key) {
            key_released = 0; last_key = pressed;
            switch(pressed) {
                case 11: /* '#' - confirm */
                    SW_Tx_UART_PutCRLF();
                    checkPassword();
                    break;
                case 10: /* '*' - backspace */
                    if(inputLen > 0) { inputLen--;
printInputLine(); }
                    break;
                default:
                    if(inputLen < MAX_INPUT_LEN) {
                        inputBuf[inputLen++] = pressed;
                        SW_Tx_UART_PutString("*");
                    }
                    break;
            }
        }
    }
}
```

4.2. Результат виконання в консолі (Завдання 3)

Рисунок 15 – Правильний пароль: "Access allowed"

Рисунок 16 – Неправильний пароль: "Access denied"

5. Висновки

У ході виконання лабораторної роботи було ознайомлено з принципами підключення та програмного опитування матричної клавіатури 4×3 у середовищі PSoC® Creator™.

У завданні 1 реалізовано керування кольором RGB-світлодіода за допомогою конструкції if-else: кожна кнопка змінює колір відповідно до таблиці (Red, Green, Blue, Yellow, Purple, Cyan), а при відпусканні кнопки вмикається білий колір.

У завданні 2 реалізовано аналогічний алгоритм з використанням конструкції switch(case), що робить код більш структурованим і зручним для розширення.

У завданні 3 реалізовано систему введення паролю: клавіша * виконує Backspace, клавіша # – підтвердження. Введення виводиться у вигляді зірочок (*), а результат перевірки відображається у терміналі як "Access allowed" або "Access denied". Успішне введення підсвічується зеленим LED, невдале – червоним.

Набуто практичних навичок роботи з матричними клавіатурами, масивами вказівників на функції, а також організації введення даних у вбудованих системах.